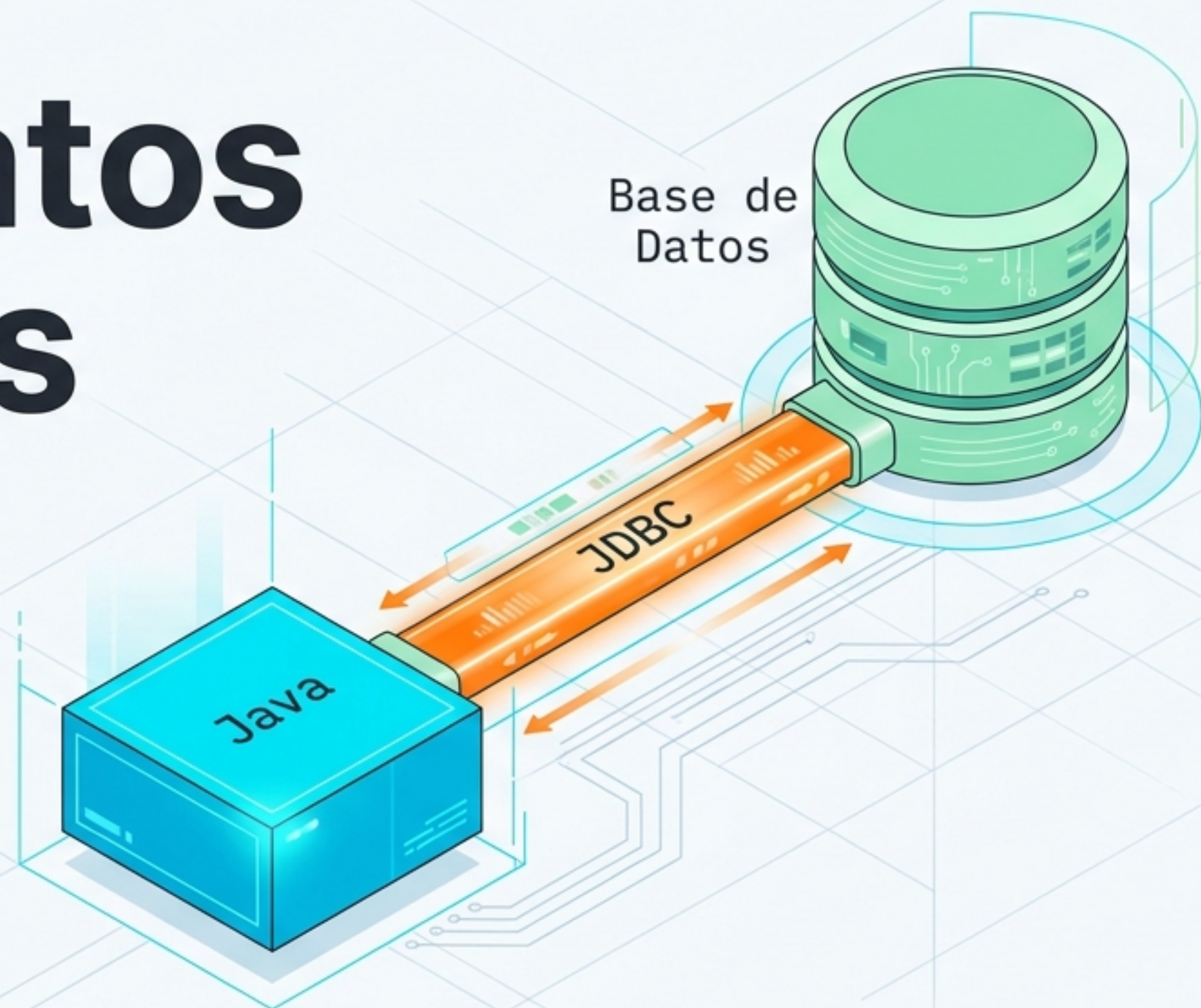


Bases de Datos Relacionales y JDBC

Arquitectura, conexión y manipulación de datos en Java



La evolución del almacenamiento: De ficheros a bases de datos

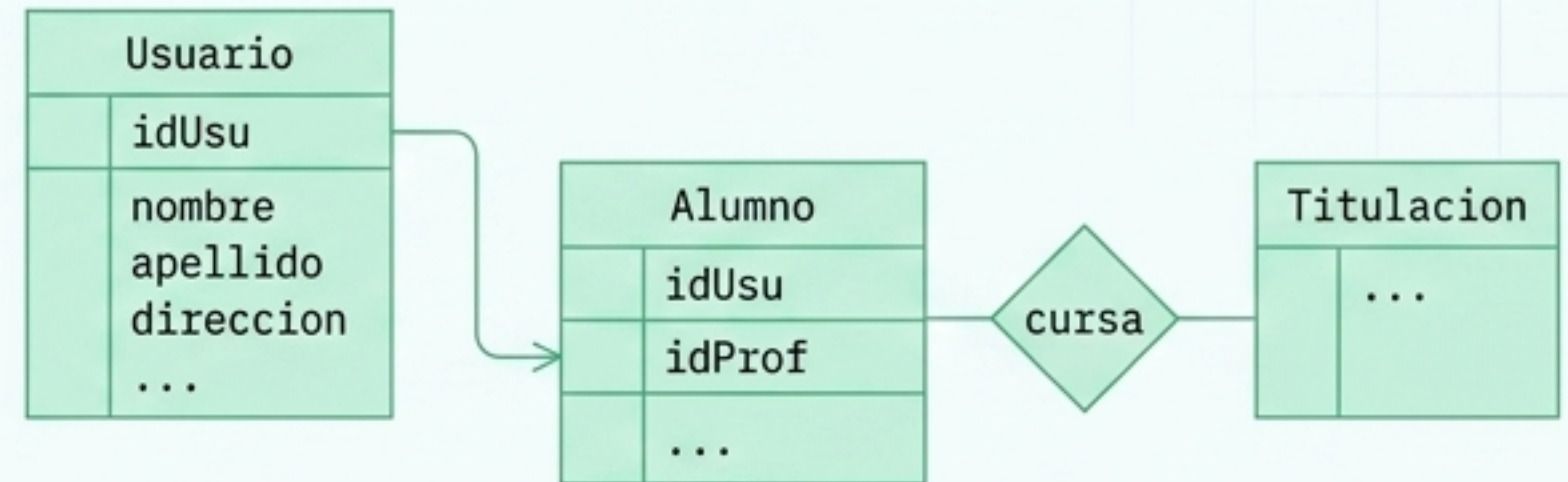
El Pasado: Ficheros



Acceso Secuencial

Lentos y secuenciales. Ineficientes para grandes volúmenes de datos o búsquedas complejas.

El Presente: BD Relacionales



- Estructura: Datos organizados en filas (entidades) y columnas (atributos).
- Relación: Tablas vinculadas matemáticamente mediante claves.
- Normalización: Diseño que elimina la redundancia de datos.
- Escalabilidad: Crecimiento seguro vertical y horizontal.

Ecosistema SGBD: ¿Qué base de datos elegir?

Resolución: Caso Práctico 1

SGBD	Tipo	Características	Caso de Uso Ideal
MySQL 	Relacional	Veloz, fiable, Open Source	Aplicaciones web estándar
 PostgreSQL	Relacional	Robusto, avanzado, Open Source	Sistemas complejos y estándares estrictos
ORACLE®	Relacional	Máxima seguridad y rendimiento	Entornos corporativos masivos
 SQLite	Relacional	Ligero, sin servidor dedicado	Apps móviles y sistemas embebidos
 Microsoft SQL Server	Relacional	Integración ecosistema Microsoft	Entornos empresariales Windows
 mongoDB	No Relacional	Almacenamiento JSON, esquemas flexibles	Manejo de documentos, ecosistemas NoSQL

El puente tecnológico: Arquitectura de la API JDBC



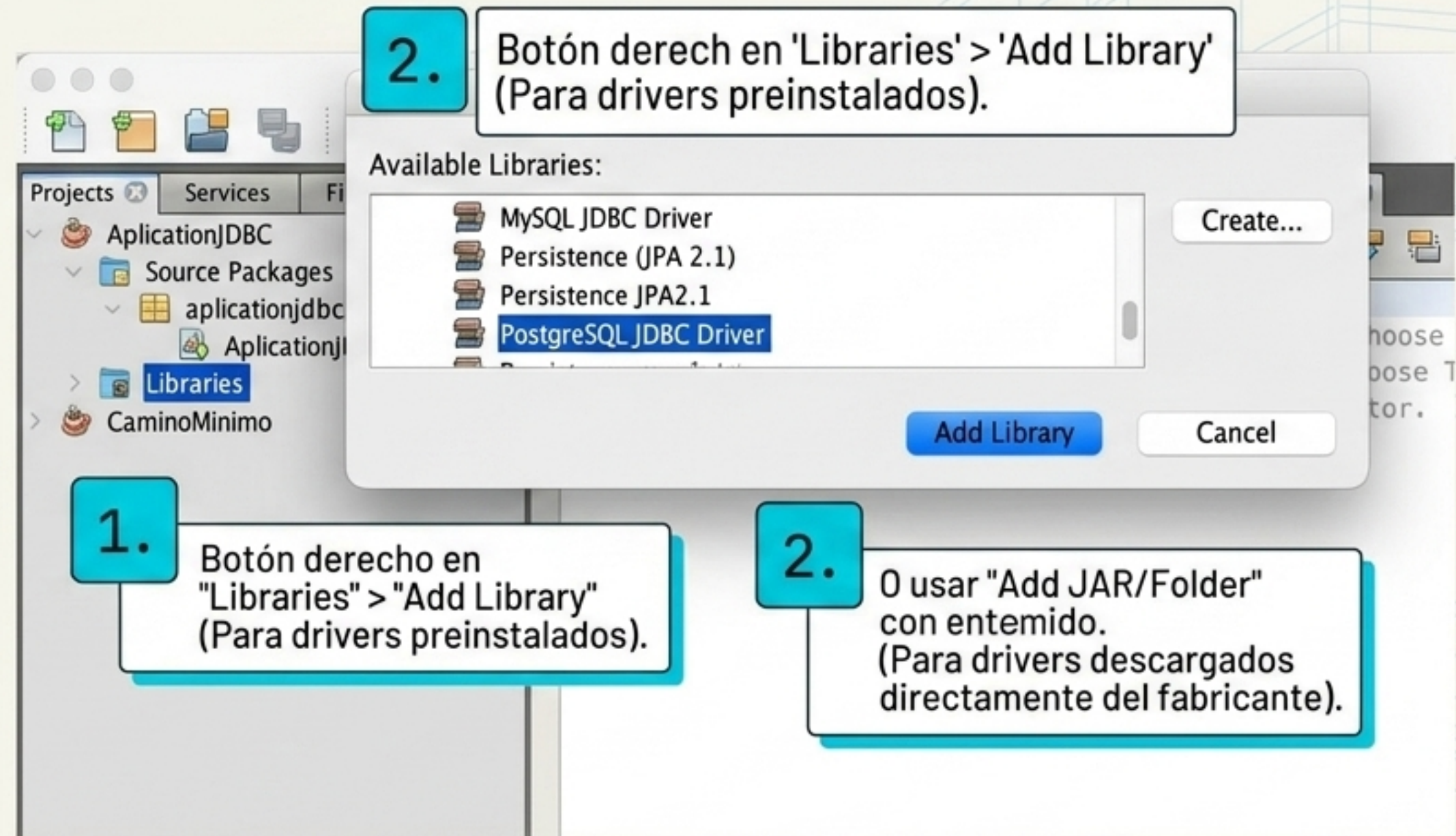
La API JDBC no conoce los detalles del SGBD. Delega la comunicación al Driver específico, permitiendo cambiar de base de datos sin reescribir la lógica de tu aplicación Java.

Paso 1: Integración del Driver específico



El Traductor del SGBD

- **¿Qué es?** Componente de software que implementa las interfaces de JDBC para un SGBD concreto (ej. MySQL, PostgreSQL).
- **¿Qué provee?** Clases vitales como `Connection`, `Statement` y `ResultSet`.



Paso 2: Estableciendo el túnel de comunicación



Protocolo principal

Servidor (IP de la máquina)

Nombre de la base de datos

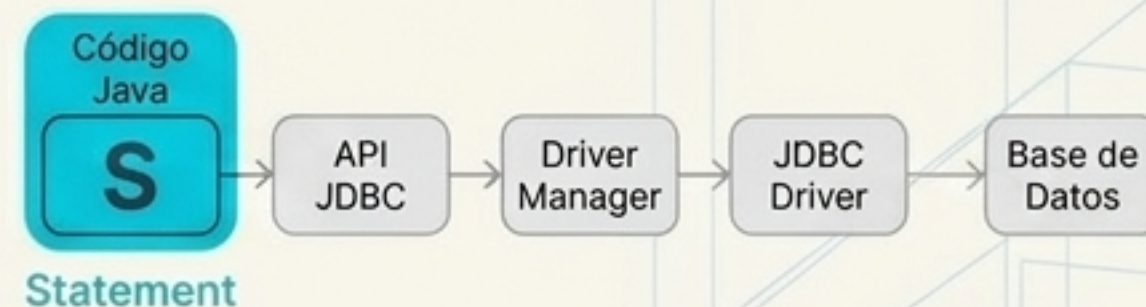
```
String url = "jdbc:mysql://localhost:3306/MedacBD";  
Connection conn = DriverManager.getConnection(url, "usuario", "password");
```

Subprotocolo
(identifica el SGBD)

Servidor
(IP de la máquina)

Importante: Toda conexión debe cerrarse obligatoriamente usando **conn.close()** para liberar recursos del sistema.

Paso 3: El motor de ejecución (Statement)



Modificación de Datos (CRUD)

```
stmt.executeUpdate(sql);
```

Devuelve: int
(nº de filas afectadas)

```
Statement stmt =  
connection.createStatement();
```

Consulta de Datos (SELECT)

```
stmt.executeQuery(sql);
```

Devuelve: ResultSet
(Paquete de datos)

Atención a la Sintaxis

En Java, utiliza la secuencia de escape `"` para introducir comillas dobles, o usa comillas simples `' '` dentro del String para envolver valores de texto en SQL.

Operaciones de Modificación: executeUpdate()

Comando SQL	Ejecución Java	Resultado
INSERT INTO Alumno (DNI, nombre) VALUES ('147A', 'Pepe')	<code>int</code> filas = <code>stmt.executeUpdate(sql);</code>	Añade una nueva entidad. (Retorna 1)
UPDATE persona SET edad = '20' WHERE nombre LIKE 'F%'	<code>int</code> filas = <code>stmt.executeUpdate(sql);</code>	Modifica registros existentes que cumplen la condición.
DELETE FROM persona WHERE edad >= '18'	<code>int</code> filas = <code>stmt.executeUpdate(sql);</code>	Elimina registros. ¡Peligro: Requiere cláusula WHERE o vaciará la tabla entera!

Operaciones de Consulta: executeQuery()

1. Construcción SQL

```
String sql = "SELECT * FROM persona WHERE edad >= 18";
```

2. Ejecución Java

```
ResultSet rs = stmt.executeQuery(sql);
```

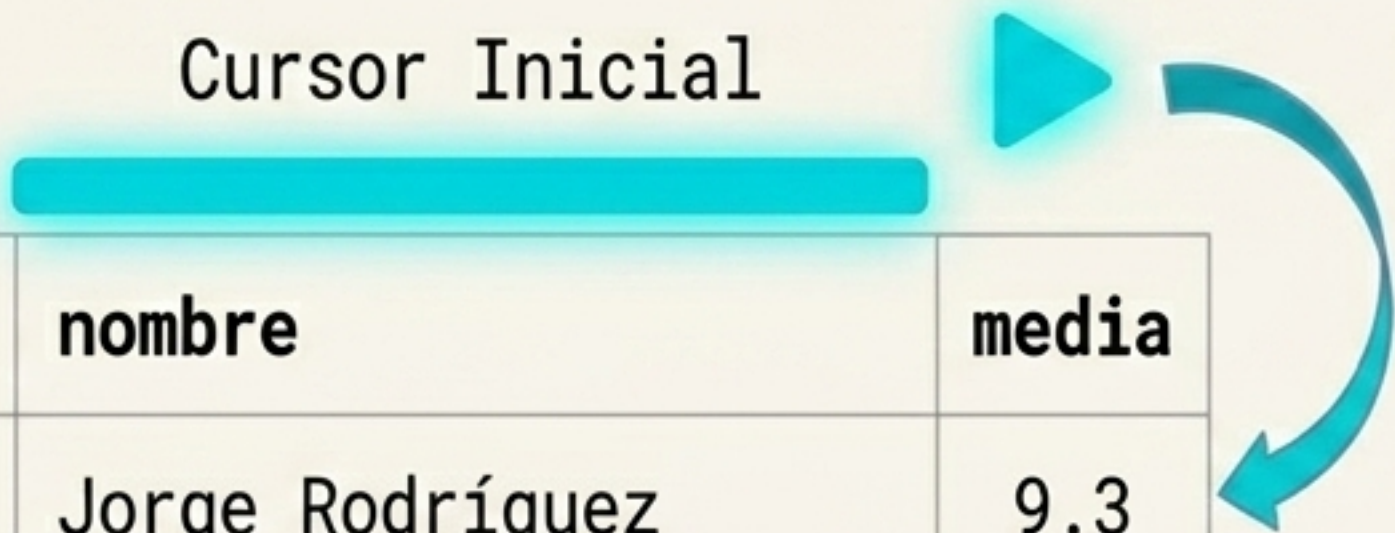
3. Emmpaquetado de Resultados

La base de datos devuelve un objeto ResultSet que contiene la tabla de datos solicitada.

A diferencia de un simple 'int', el objeto ResultSet requiere navegación manual (un cursor) para extraer los datos fila por fila hacia nuestra aplicación Java.

Procesamiento del ResultSet: El Cursor de Lectura

Cursor Inicial



num	nombre	media
1	Jorge Rodríguez	9.3
2	Francisco José García	7.6
3	Francisco Jiménez	8.9

`rs.next()`
devuelve `true`

```
while (rs.next()) {  
    String nombre = rs.getString("nombre");  
    double nota = rs.getDouble("media");  
    System.out.println(nombre);  
}
```

El bucle empuja el cursor hacia abajo fila por fila. Cuando no hay más datos, `next()` devuelve `false` y termina el bucle.

Tipología y Navegación Avanzada del ResultSet

Dirección del Cursor (Scroll)



TYPE_FORWARD_ONLY.
Secuencial hacia adelante.
Menor consumo de memoria.
(Por defecto).



TYPE_SCROLL_INSENSITIVE.
Avance, retroceso y salto.
Insensible a cambios externos
en la BD.



TYPE_SCROLL_SENSITIVE.
Bidireccional. Refleja en tiempo
real los cambios realizados por
otras conexiones.

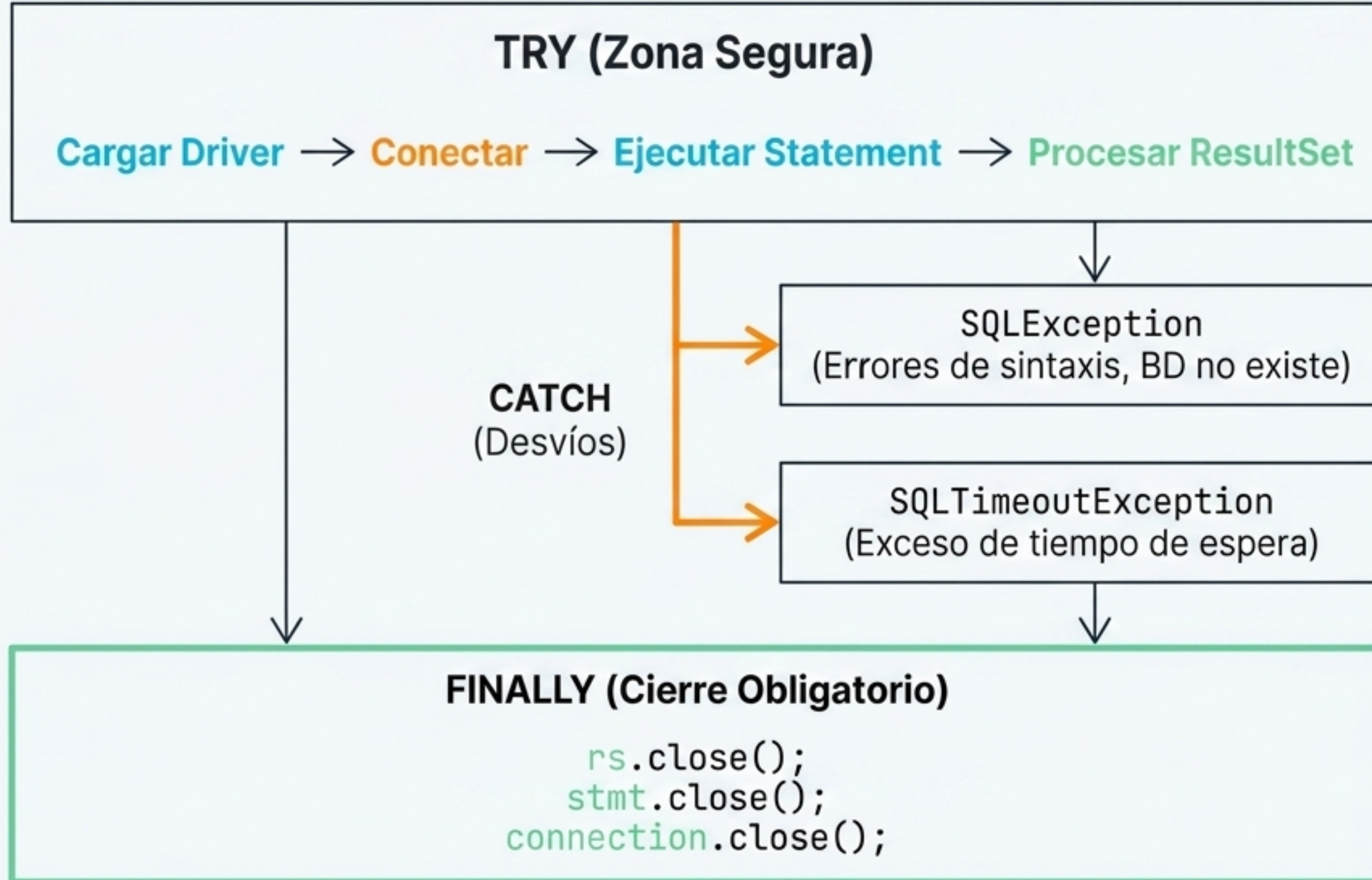
Concurrencia (Edición)

CONCUR_READ_ONLY
Cursor de solo lectura.

CONCUR_UPDATABLE
Permite modificar la base de datos
directamente desde el ResultSet.

```
createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,  
                ResultSet.CONCUR_READ_ONLY);
```

Código Robusto: Gestión del Ciclo de Vida y Excepciones

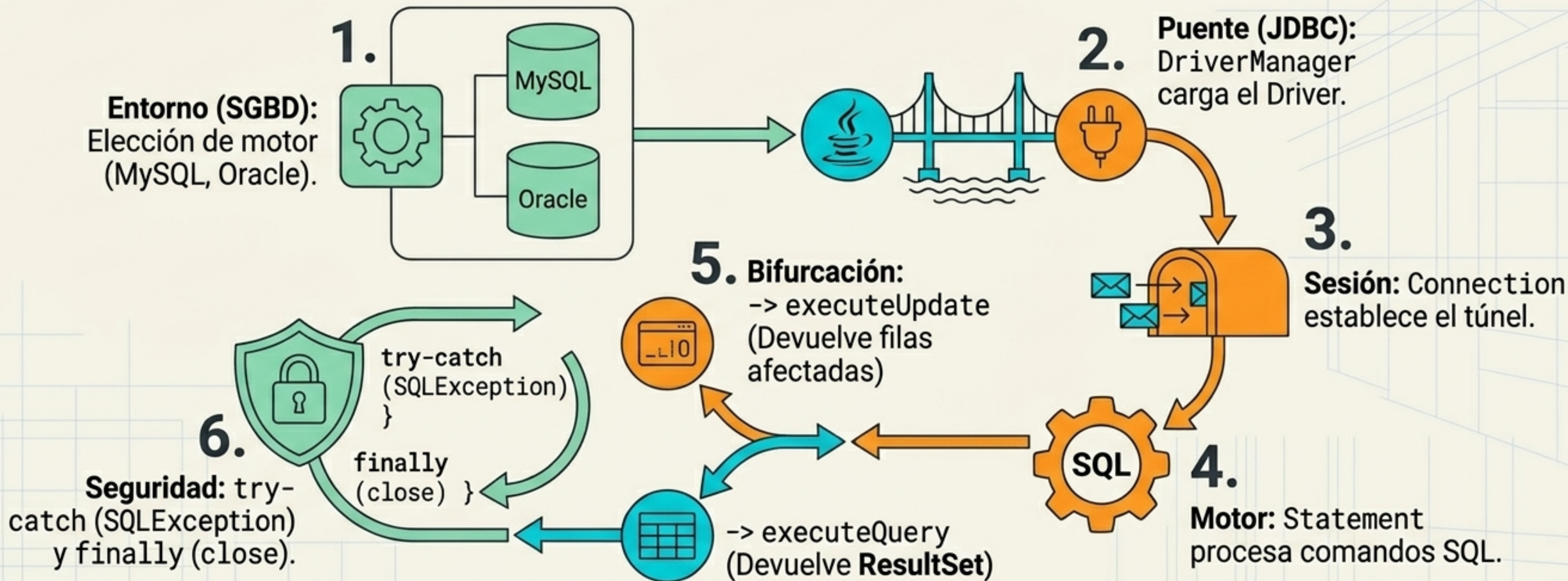


Resolución Caso Práctico 2

Para evitar repetir este bloque en cada consulta, encapsula la conexión y desconexión en una clase dedicada (Patrón DAO).

Reutiliza el código, **asegura el cierre de recursos y facilita el mantenimiento.**

Síntesis: El Ecosistema Relacional en Java



Las bases de datos relacionales y JDBC proporcionan la estructura, velocidad y fiabilidad necesarias para convertir datos crudos en información accionable dentro de cualquier aplicación Java profesional.